

2022~2023 学年春夏学期数据库系统期末考试（回忆卷）

一、选择题（每题 3 分，共 8 题）

1、对于如下定义的一个表

```
create table course (  
    course_id char(2) primary key,  
    credit int,  
    prereq_id char(2),  
    foreign key prereq_id references course(course_id)  
        on delete cascade  
        on update cascade  
);
```

其中有如下数据:

course_id	credit	prereq_id
11	2	
22	4	11
33	3	
44	3	22
55	4	22
66	3	33

在该表上执行如下 SQL 语句后，最后一条语句返回的结果是：

```
(1) insert into course('77', 4, '11');  
(2) delete from course where course_id = '11';  
(3) update course set course_id = '11' credit = 2 where course_id = '33';  
(4) select count(*) from course where credit > 2;
```

- A. 1
- B. 2
- C. 3
- D. 4

2、对于如第一题中定义的表 course，对于如下 SQL 查询语句：

```
select c1.course_id, c1.credit, c2.course_id, c2.credit  
from course as c1, course as c2  
where c1.prereq_id = c2.course_id and c1.credit = 4 and c2.credit > 2;
```

下列关系代数表达式中与上面的 SQL 查询语句不等价的是：(这道题的选项有点记不清了，写了一个差不多的)

- A. $\prod_{c1.course_id, c1.credit, c2.course_id, c2.credit} (\sigma_{c1.credit=4 \wedge c2.credit>2} (\rho_{c1}(course) \bowtie \rho_{c2}(course)))$
- B. $\prod_{c1.course_id, c1.credit, c2.course_id, c2.credit} (\sigma_{c1.credit=4 \wedge c2.credit>2 \wedge c1.prereq_id=c2.course_id} (\rho_{c1}(course) \times \rho_{c2}(course)))$
- C. $\prod_{c1.course_id, c1.credit, c2.course_id, c2.credit} (\sigma_{c1.prereq_id=c2.course_id} (\rho_{c1}(\sigma_{credit=4}(course)) \times \rho_{c2}(\sigma_{credit>2}(course))))$
- D. $\prod_{c1.course_id, c1.credit, c2.course_id, c2.credit} (\sigma_{c1.prereq_id=c2.course_id} (\rho_{c1}(\sigma_{credit=4}(course)) \times \prod_{course_id, credit} (\rho_{c2}(\sigma_{credit>2}(course)))))$

3、关系模式R(A,B,C,D,E,F)上有函数依赖关系 {A->B, B->C, C->B, D->E, D->F}。下列选项中是该关系模式的候选键的是：

- A. AC
- B. AD

C. AE

D. AF

4、函数依赖集{A->B, A->D, BC->E, AC->D}的正则覆盖是 (这道题选项记不太清了)

A. {A->BDE, BC->E}

B. {A->BD, ABC->E}

C. {A->BD, C->E}

D. {A->BD, BC->E}

5、关系 A 上有 20,000 条记录。对于关系 A 的非键属性建立 B+树主索引。每个 B+树节点的大小是 2KB，每个 B+树内键值的大小是 32B，每个指针的大小是 8B。若某次查询得到的结果包含在 5 个块中，则此次查询的开销是

A. 4 seek + 8 transfer

B. 4 seek + 9 transfer

C. 5 seek + 8 transfer

D. 5 seek + 9 transfer

6、关系 r 上有 50,000 条记录，每个块上可以容纳 r 上的 200 条记录；关系 s 上有 20,000 条记录，每个块上可以容纳 s 上的 50 条记录。在不考虑递归分块的情况下，对 r 和 s 进行 hash join 的开销约为

A. 1950 transfer + 1350 seek

B. 1950 transfer + 650 seek

C. 3300 transfer + 1350 seek

D. 3300 transfer + 650 seek

7、关系 r 上有 30,000 条记录，每个块上可以容纳 200 条记录。若在 r 的候选键上进行等值查找，则返回的结果的大小约为

A. 1

B. 20

C. 1,500

D. 30,000

8、在下面的执行计划中，lock-S(A)表示给 A 加上共享锁，lock-X(B)表示给 B 加上互斥锁。(这道题的执行计划有点记不清了，写了一个差不多的)

T1	T2	T3
lock-S(A)		
	lock-X(A)	
	lock-X(B)	
		lock-X(B)
		lock-S(C)
lock-X(C)		

对于上面的执行计划，下列说法正确的是：

A. 计划中没有死锁

B. 计划中有死锁，T1 在等待 T2 放锁的同时 T2 也在等待 T1 放锁

C. 计划中有死锁，T2 在等待 T3 放锁的同时 T3 也在等待 T2 放锁

D. 计划中有死锁，T2 在等待 T1 放锁，T3 在等待 T2 放锁，T1 在等待 T3 放锁

二、判断题（每题 2 分，共 6 题）

1. 任意关系模式均可分解为三范式，且分解是保持依赖的。

2. 在事务频繁更新数据的场景下，按列存储的效率比按行存储高。
3. 建立辅助索引的时候，为了降低空间开销，可以对每一个数据块建立稀疏索引。
4. 连接前尽可能早地进行投影操作可以降低投影操作的开销，所以投影操作应该尽早做。
5. 按索引查找的时间复杂度总是比顺序查找低。
6. 不能通过 2PL 产生的调度，可能可以由树协议产生。

三、SQL 语句（16 分）

对于如下定义的关系模式

user(userID, name, gender, age, group_name)

followship(userID, followerID)

写出可以进行如下查询的 SQL 语句：

- (1) 查询所有含有男性成员的群组（group）
- (2) 查询 userID 为'1001'的用户的追随者的信息
- (3) 查出女性成员最多的群组
- (4) 查询名称为'game'的群组当中追随者（follower）数量高于该群组中所有用户的追随者数量的平均值的用户。

四、数据库设计（16 分）

- (1) 请根据以下需求，为跟团旅行预订平台设计 E-R 图
 - 每个用户都有用户 ID、用户名、密码和联系方式
 - 每个旅行社需要提供旅行社的名称、法人代表身份证号、企业银行账号、位置，同时平台会给每个旅行社分配 ID
 - 旅行社会出售旅行计划，旅行计划的内容包含标题、时长、描述
 - 每个用户都可以预订旅行计划，订单信息包含用户信息、旅行计划信息、订单发起时间、旅行计划数量、总价格、支付状态
 - 旅行结束后用户可以对旅行计划进行评价
- (2) 请将第 1 问设计的 E-R 图转化为关系模式，并指明主键和外键

五、并发控制（8 分）

对于如下的访问序列：（题目中具体的序列记不清了，凭印象写了一个和题目差不多的）

r1(A) w2(A) w2(B) w3(A) r1(A) w3(B) r4(C) r4(B) w5(C) w5(B) r3(C)

注：其中 $r_i(D)$ 表示事务 T_i 对数据 D 进行读操作， $w_i(D)$ 表示事务 T_i 对数据 D 进行写操作。

- (1) 根据访问序列画出各个事务的前驱图
- (2) 判断这一访问序列是否冲突可串行化并说明理由
- (3) 判断这一访问序列是否可以由两阶段锁协议产生并说明理由

六、ARIES 恢复算法（12 分）

给出如下的 ARIES 日志：（题目中具体的日志记不清了，这里写了一个差不多的）

8001: <T1, begin>
 8002: <T1, 3025.1, 20, 40>
 8003: <T2, begin>
 8004: <T2, 2628.2, 55, 555>
 8005: <T3, begin>
 8006: <T3, 2628.1, 30, 60>
 8007: <T2, commit>
 8008: <checkpoint>

Transaction		LastLSN
T1		8002
T3		8006

PageID	PageLSN	RecLSN
3025	8002	8002
2628	8006	8004

8009: <T4, begin>
 8010: <T3, 2628.1, 60, 90>
 8011: <T4, 6601.1, 30, 40>
 8012: <T3, 2628.2, 555, 550>
 8013: <T1, commit>
 8014: <T4, 6601.2, 20, 30>

- (1) 写出在分析过程中被加入到脏页表的记录
- (2) 重做过程从哪一条日志开始
- (3) 写出恢复过程中增加的日志
- (4) 恢复过程结束后，6601.1和2628.2的值分别是多少

七、Buffer Tree (12 分)

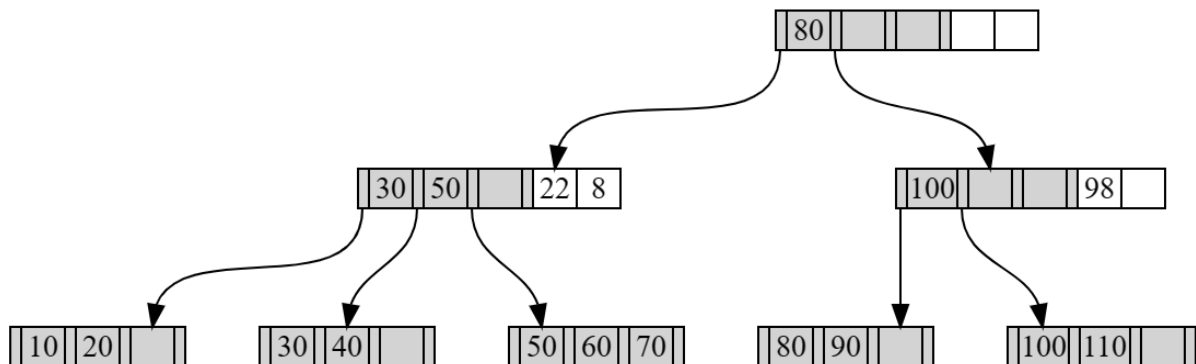
Buffer Tree 是一种写优化的索引结构。即在 B+树的每个内节点出增加一块缓冲区。

当进行插入操作的时候，我们直接将要插入的键值写入 Buffer Tree 根节点的缓冲区中。

当某一节点的缓冲区溢出的时候：

- 若该节点的子节点为内节点，则将该节点的缓冲区中的所有元素写入其子节点的缓冲区中。
- 若该节点的子节点为叶子，则将该节点的所有元素插入其子节点，当其子节点溢出的时候正常进行分裂。

本题中我们讨论缓冲区的大小为 2 且度为 4 的 Buffer Tree。其初始状态如下图：



- (1) 现对该树范围查询不大于 30、不小于 50 的键值，该过程中有多少个节点被访问？
- (2) 向该树中依次插入三个键值：38、92、28。请画出三次插入操作之后的 Buffer Tree。
- (3) 在第 (2) 问的插入操作中，每一次插入有多少节点被修改？

注：此处根节点始终被缓存在内存之中，所以计算时忽略根节点；分裂时新增加的节点视同被修改的节点。